ADA_boost

October 22, 2026

[1]:
```
import pandas as pd
import numpy as np
import pickle
```

[2]:
```
dataset = pd.read_csv("insurance.csv")
dataset
```

[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
…	…	…	…	…	…	…	…
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

[1338 rows x 7 columns]

[3]:
```
dataset =pd.get_dummies(dataset,drop_first= True)
dataset
```

[3]:

	age	bmi	children	charges	sex_male	smoker_yes	\
0	19	27.900	0	16884.92400	False	True	
1	18	33.770	1	1725.55230	True	False	
2	28	33.000	3	4449.46200	True	False	
3	33	22.705	0	21984.47061	True	False	
4	32	28.880	0	3866.85520	True	False	
…	…	…	…	…	…	…	
1333	50	30.970	3	10600.54830	True	False	
1334	18	31.920	0	2205.98080	False	False	
1335	18	36.850	0	1629.83350	False	False	
1336	21	25.800	0	2007.94500	False	False	
1337	61	29.070	0	29141.36030	False	True	

```
      region_northwest  region_southeast  region_southwest
0                False             False              True
1                False              True             False
2                False              True             False
3                 True             False             False
4                 True             False             False
…                 …                …                 …
1333              True             False             False
1334             False             False             False
1335             False              True             False
1336             False             False              True
1337              True             False             False

[1338 rows x 9 columns]
```

[6]:
```
x = dataset[["age","bmi","children","smoker_yes","sex_male"]]
y = dataset[["charges"]]
```

[7]:
```
y
```

[7]:
```
          charges
0     16884.92400
1      1725.55230
2      4449.46200
3     21984.47061
4      3866.85520
…             …
1333  10600.54830
1334   2205.98080
1335   1629.83350
1336   2007.94500
1337  29141.36030

[1338 rows x 1 columns]
```

[8]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 1/
 ↪3,random_state =0 )
```

[9]:
```
from sklearn.ensemble import AdaBoostRegressor
regr = AdaBoostRegressor(random_state=0, n_estimators=100)
regr.fit(x_train, y_train)
```

```
C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\utils\validation.py:1368: DataConversionWarning: A column-
vector y was passed when a 1d array was expected. Please change the shape of y
to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

[9]:
```
AdaBoostRegressor(n_estimators=100, random_state=0)
```

[11]:
```
y_pred = regr.predict(x_test)
y_pred
```

[11]:
```
array([14059.47430713, 11450.80214932, 45121.15854841, 14678.67167687,
       11450.80214932,  7639.2207935 ,  3986.63853774, 13263.79698437,
        9141.00750357,  9141.00750357,  7639.2207935 , 13263.79698437,
       11450.80214932,  7796.99159733, 20436.74816106, 13263.79698437,
       16705.68320797,  9141.00750357,  9141.00750357, 40942.4943172 ,
       25627.89524677, 19799.50996132, 14059.47430713, 26988.8426    ,
        3986.63853774,  9141.00750357,  5421.58225157,  9141.00750357,
        7796.99159733, 11450.80214932,  9141.00750357, 45121.15854841,
       16705.68320797, 14059.47430713, 20436.74816106,  7796.99159733,
       11450.80214932, 40942.4943172 , 44527.54166392,  3986.63853774,
        7639.2207935 ,  7796.99159733, 20436.74816106, 45121.15854841,
       40942.4943172 ,  3986.63853774, 13263.79698437,  9141.00750357,
        5421.58225157, 14059.47430713,  9141.00750357,  3986.63853774,
       26988.8426    , 45121.15854841, 14059.47430713,  9141.00750357,
        9141.00750357, 13123.95538509, 11450.80214932, 16705.68320797,
        3986.63853774, 45121.15854841, 19799.50996132, 13263.79698437,
       13263.79698437, 13263.79698437, 36927.10711852, 44527.54166392,
        3986.63853774,  9141.00750357, 16705.68320797, 14059.47430713,
       18915.9257486 , 16705.68320797, 16705.68320797, 16705.68320797,
       11450.80214932, 11450.80214932, 20436.74816106, 45121.15854841,
       14059.47430713, 45121.15854841,  3986.63853774, 11450.80214932,
       44527.54166392, 20436.74816106,  9141.00750357,  9141.00750357,
       16705.68320797, 44527.54166392,  9141.00750357, 14059.47430713,
        9141.00750357, 11450.80214932,  9141.00750357,  9141.00750357,
       45121.15854841, 45121.15854841,  9141.00750357, 14059.47430713,
        9141.00750357,  9141.00750357,  9141.00750357, 44527.54166392,
       25019.11425986,  5421.58225157, 14059.47430713,  9141.00750357,
       14059.47430713, 45121.15854841, 19799.50996132,  9141.00750357,
       13263.79698437, 40942.4943172 , 45121.15854841,  9141.00750357,
        3986.63853774, 13123.95538509, 16705.68320797, 11450.80214932,
       44527.54166392, 14059.47430713, 14678.67167687,  9141.00750357,
       11450.80214932,  5421.58225157, 26988.8426    , 25627.89524677,
       45121.15854841,  3986.63853774, 11450.80214932,  3986.63853774,
        9141.00750357, 11450.80214932, 25019.11425986, 44527.54166392,
       25019.11425986, 13263.79698437, 25627.89524677,  9141.00750357,
        3986.63853774, 14059.47430713, 13263.79698437, 14059.47430713,
        9141.00750357, 16705.68320797, 16705.68320797,  9141.00750357,
        3986.63853774, 14059.47430713,  9141.00750357, 40942.4943172 ,
       13263.79698437,  3986.63853774,  3986.63853774,  9141.00750357,
        3986.63853774,  3986.63853774, 13263.79698437, 11450.80214932,
        9141.00750357, 14678.67167687, 16705.68320797, 13263.79698437,
        5421.58225157,  9141.00750357, 18915.9257486 ,  9141.00750357,
```

 9141.00750357, 3986.63853774, 9141.00750357, 9141.00750357,
11450.80214932, 9141.00750357, 9141.00750357, 9141.00750357,
 5421.58225157, 7639.2207935 , 26988.8426   , 3986.63853774,
19799.50996132, 9141.00750357, 14059.47430713, 9141.00750357,
 7639.2207935 , 26988.8426   , 3986.63853774, 3986.63853774,
16705.68320797, 13263.79698437, 45121.15854841, 9141.00750357,
 9141.00750357, 25627.89524677, 9141.00750357, 7796.99159733,
 9141.00750357, 9141.00750357, 9141.00750357, 9141.00750357,
14059.47430713, 45121.15854841, 19799.50996132, 20436.74816106,
 9141.00750357, 45121.15854841, 9141.00750357, 14059.47430713,
 9141.00750357, 3986.63853774, 13263.79698437, 16705.68320797,
 9141.00750357, 9141.00750357, 9141.00750357, 7796.99159733,
 9141.00750357, 7639.2207935 , 19799.50996132, 7639.2207935 ,
 9141.00750357, 9141.00750357, 13123.95538509, 7796.99159733,
14059.47430713, 14678.67167687, 19799.50996132, 14059.47430713,
 7639.2207935 , 7796.99159733, 3986.63853774, 11450.80214932,
16705.68320797, 3986.63853774, 9141.00750357, 9141.00750357,
13263.79698437, 40942.4943172 , 9141.00750357, 19799.50996132,
 7796.99159733, 40942.4943172 , 9141.00750357, 11450.80214932,
11450.80214932, 14059.47430713, 3986.63853774, 11450.80214932,
 3986.63853774, 9141.00750357, 20436.74816106, 45121.15854841,
 9141.00750357, 9141.00750357, 3986.63853774, 3986.63853774,
11450.80214932, 3986.63853774, 9141.00750357, 9141.00750357,
11450.80214932, 26988.8426   , 45121.15854841, 19799.50996132,
 9141.00750357, 14059.47430713, 44527.54166392, 11450.80214932,
44527.54166392, 3986.63853774, 40942.4943172 , 9141.00750357,
14059.47430713, 9141.00750357, 44527.54166392, 9141.00750357,
14059.47430713, 11450.80214932, 3986.63853774, 16705.68320797,
14059.47430713, 9141.00750357, 16705.68320797, 3986.63853774,
 9141.00750357, 9141.00750357, 9141.00750357, 25627.89524677,
 9141.00750357, 3986.63853774, 3986.63853774, 45121.15854841,
13263.79698437, 9141.00750357, 19799.50996132, 14059.47430713,
40942.4943172 , 9141.00750357, 9141.00750357, 13123.95538509,
 3986.63853774, 3986.63853774, 20436.74816106, 20436.74816106,
16705.68320797, 7796.99159733, 9141.00750357, 11450.80214932,
14059.47430713, 18915.9257486 , 20436.74816106, 9141.00750357,
18915.9257486 , 3986.63853774, 9141.00750357, 9141.00750357,
11450.80214932, 3986.63853774, 11450.80214932, 44527.54166392,
 3986.63853774, 9141.00750357, 3986.63853774, 13123.95538509,
 9141.00750357, 14059.47430713, 45121.15854841, 20436.74816106,
13263.79698437, 44527.54166392, 19799.50996132, 9141.00750357,
13263.79698437, 9141.00750357, 26988.8426   , 9141.00750357,
16705.68320797, 3986.63853774, 18915.9257486 , 5421.58225157,
25627.89524677, 9141.00750357, 7796.99159733, 40942.4943172 ,
 9141.00750357, 7796.99159733, 26988.8426   , 3986.63853774,
11450.80214932, 3986.63853774, 13263.79698437, 5421.58225157,
 9141.00750357, 9141.00750357, 45121.15854841, 13123.95538509,

```
45121.15854841, 40942.4943172 ,  9141.00750357,  9141.00750357,
 3986.63853774,  7796.99159733,  3986.63853774, 13263.79698437,
45121.15854841, 14059.47430713,  3986.63853774, 16705.68320797,
 3986.63853774, 11450.80214932,  9141.00750357,  9141.00750357,
 9141.00750357, 40942.4943172 ,  7639.2207935 ,  3986.63853774,
44527.54166392, 14059.47430713, 45121.15854841, 14059.47430713,
 9141.00750357,  5421.58225157,  3986.63853774, 13263.79698437,
 9141.00750357,  3986.63853774, 14059.47430713,  9141.00750357,
 7796.99159733,  9141.00750357,  7796.99159733, 44527.54166392,
 3986.63853774, 13123.95538509,  3986.63853774,  9141.00750357,
14059.47430713,  9141.00750357, 18915.9257486 , 14059.47430713,
16705.68320797, 16705.68320797,  3986.63853774,  9141.00750357,
19799.50996132, 11450.80214932,  9141.00750357,  3986.63853774,
40942.4943172 , 16705.68320797,  9141.00750357, 14678.67167687,
11450.80214932, 19799.50996132, 45121.15854841,  3986.63853774,
11450.80214932,  3986.63853774, 19799.50996132,  3986.63853774,
 7639.2207935 ,  3986.63853774, 14059.47430713,  9141.00750357,
14059.47430713, 45121.15854841,  9141.00750357, 40942.4943172 ,
20436.74816106, 14059.47430713, 11450.80214932, 40942.4943172 ,
13123.95538509,  7796.99159733, 13263.79698437, 25627.89524677,
40942.4943172 , 20436.74816106, 13123.95538509, 44527.54166392,
 9141.00750357, 18915.9257486 ,  9141.00750357, 40942.4943172 ,
 9141.00750357, 19530.78879845])
```

[12]:
```
from sklearn.metrics import r2_score
score =r2_score(y_test,y_pred)
score
```

[12]:
```
0.8494579300295322
```

[ ]:
```

```

XG_BOOST

October 22, 2026

[1]:
```
import pandas as pd
import numpy as np
import pickle
```

[2]:
```
dataset = pd.read_csv("insurance.csv")
dataset
```

[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
…	…	…	…	…	…	…	…
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

[1338 rows x 7 columns]

[3]:
```
dataset =pd.get_dummies(dataset,drop_first= True)
dataset
```

[3]:

	age	bmi	children	charges	sex_male	smoker_yes	\
0	19	27.900	0	16884.92400	False	True	
1	18	33.770	1	1725.55230	True	False	
2	28	33.000	3	4449.46200	True	False	
3	33	22.705	0	21984.47061	True	False	
4	32	28.880	0	3866.85520	True	False	
…	…	…	…	…	…	…	
1333	50	30.970	3	10600.54830	True	False	
1334	18	31.920	0	2205.98080	False	False	
1335	18	36.850	0	1629.83350	False	False	
1336	21	25.800	0	2007.94500	False	False	
1337	61	29.070	0	29141.36030	False	True	

```
      region_northwest  region_southeast  region_southwest
0                False             False              True
1                False              True             False
2                False              True             False
3                 True             False             False
4                 True             False             False
…                 …                …                 …
1333              True             False             False
1334             False             False             False
1335             False              True             False
1336             False             False              True
1337              True             False             False

[1338 rows x 9 columns]
```

[6]:
```
x = dataset[["age","bmi","children","smoker_yes","sex_male"]]
y = dataset[["charges"]]
```

[7]:
```
y
```

[7]:
```
          charges
0     16884.92400
1      1725.55230
2      4449.46200
3     21984.47061
4      3866.85520
…             …
1333  10600.54830
1334   2205.98080
1335   1629.83350
1336   2007.94500
1337  29141.36030

[1338 rows x 1 columns]
```

[13]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 1/
 ↪3,random_state =0 )
```

[15]:
```
from sklearn.ensemble import GradientBoostingRegressor
reg =␣
 ↪GradientBoostingRegressor(criterion='deprecated',random_state=0,verbose=0,loss='squared_err
reg.fit(x_train, y_train)
```

```
C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\ensemble\_gb.py:691: DataConversionWarning: A column-vector y
was passed when a 1d array was expected. Please change the shape of y to
(n_samples, ), for example using ravel().
```

```
    y = column_or_1d(y, warn=True)  # TODO: Is this still required?
```

[15]:
```
GradientBoostingRegressor(random_state=0)
```

[16]:
```
y_pred = regr.predict(x_test)
y_pred
```

[16]:
```
array([14059.47430713, 11450.80214932, 45121.15854841, 14678.67167687,
       11450.80214932,  7639.2207935 ,  3986.63853774, 13263.79698437,
        9141.00750357,  9141.00750357,  7639.2207935 , 13263.79698437,
       11450.80214932,  7796.99159733, 20436.74816106, 13263.79698437,
       16705.68320797,  9141.00750357,  9141.00750357, 40942.4943172 ,
       25627.89524677, 19799.50996132, 14059.47430713, 26988.8426    ,
        3986.63853774,  9141.00750357,  5421.58225157,  9141.00750357,
        7796.99159733, 11450.80214932,  9141.00750357, 45121.15854841,
       16705.68320797, 14059.47430713, 20436.74816106,  7796.99159733,
       11450.80214932, 40942.4943172 , 44527.54166392,  3986.63853774,
        7639.2207935 ,  7796.99159733, 20436.74816106, 45121.15854841,
       40942.4943172 ,  3986.63853774, 13263.79698437,  9141.00750357,
        5421.58225157, 14059.47430713,  9141.00750357,  3986.63853774,
       26988.8426    , 45121.15854841, 14059.47430713,  9141.00750357,
        9141.00750357, 13123.95538509, 11450.80214932, 16705.68320797,
        3986.63853774, 45121.15854841, 19799.50996132, 13263.79698437,
       13263.79698437, 13263.79698437, 36927.10711852, 44527.54166392,
        3986.63853774,  9141.00750357, 16705.68320797, 14059.47430713,
       18915.9257486 , 16705.68320797, 16705.68320797, 16705.68320797,
       11450.80214932, 11450.80214932, 20436.74816106, 45121.15854841,
       14059.47430713, 45121.15854841,  3986.63853774, 11450.80214932,
       44527.54166392, 20436.74816106,  9141.00750357,  9141.00750357,
       16705.68320797, 44527.54166392,  9141.00750357, 14059.47430713,
        9141.00750357, 11450.80214932,  9141.00750357,  9141.00750357,
       45121.15854841, 45121.15854841,  9141.00750357, 14059.47430713,
        9141.00750357,  9141.00750357,  9141.00750357, 44527.54166392,
       25019.11425986,  5421.58225157, 14059.47430713,  9141.00750357,
       14059.47430713, 45121.15854841, 19799.50996132,  9141.00750357,
       13263.79698437, 40942.4943172 , 45121.15854841,  9141.00750357,
        3986.63853774, 13123.95538509, 16705.68320797, 11450.80214932,
       44527.54166392, 14059.47430713, 14678.67167687,  9141.00750357,
       11450.80214932,  5421.58225157, 26988.8426    , 25627.89524677,
       45121.15854841,  3986.63853774, 11450.80214932,  3986.63853774,
        9141.00750357, 11450.80214932, 25019.11425986, 44527.54166392,
       25019.11425986, 13263.79698437, 25627.89524677,  9141.00750357,
        3986.63853774, 14059.47430713, 13263.79698437, 14059.47430713,
        9141.00750357, 16705.68320797, 16705.68320797,  9141.00750357,
        3986.63853774, 14059.47430713,  9141.00750357, 40942.4943172 ,
       13263.79698437,  3986.63853774,  3986.63853774,  9141.00750357,
        3986.63853774,  3986.63853774, 13263.79698437, 11450.80214932,
        9141.00750357, 14678.67167687, 16705.68320797, 13263.79698437,
```

 5421.58225157, 9141.00750357, 18915.9257486 , 9141.00750357,
 9141.00750357, 3986.63853774, 9141.00750357, 9141.00750357,
11450.80214932, 9141.00750357, 9141.00750357, 9141.00750357,
 5421.58225157, 7639.2207935 , 26988.8426   , 3986.63853774,
19799.50996132, 9141.00750357, 14059.47430713, 9141.00750357,
 7639.2207935 , 26988.8426   , 3986.63853774, 3986.63853774,
16705.68320797, 13263.79698437, 45121.15854841, 9141.00750357,
 9141.00750357, 25627.89524677, 9141.00750357, 7796.99159733,
 9141.00750357, 9141.00750357, 9141.00750357, 9141.00750357,
14059.47430713, 45121.15854841, 19799.50996132, 20436.74816106,
 9141.00750357, 45121.15854841, 9141.00750357, 14059.47430713,
 9141.00750357, 3986.63853774, 13263.79698437, 16705.68320797,
 9141.00750357, 9141.00750357, 9141.00750357, 7796.99159733,
 9141.00750357, 7639.2207935 , 19799.50996132, 7639.2207935 ,
 9141.00750357, 9141.00750357, 13123.95538509, 7796.99159733,
14059.47430713, 14678.67167687, 19799.50996132, 14059.47430713,
 7639.2207935 , 7796.99159733, 3986.63853774, 11450.80214932,
16705.68320797, 3986.63853774, 9141.00750357, 9141.00750357,
13263.79698437, 40942.4943172 , 9141.00750357, 19799.50996132,
 7796.99159733, 40942.4943172 , 9141.00750357, 11450.80214932,
11450.80214932, 14059.47430713, 3986.63853774, 11450.80214932,
 3986.63853774, 9141.00750357, 20436.74816106, 45121.15854841,
 9141.00750357, 9141.00750357, 3986.63853774, 3986.63853774,
11450.80214932, 3986.63853774, 9141.00750357, 9141.00750357,
11450.80214932, 26988.8426   , 45121.15854841, 19799.50996132,
 9141.00750357, 14059.47430713, 44527.54166392, 11450.80214932,
44527.54166392, 3986.63853774, 40942.4943172 , 9141.00750357,
14059.47430713, 9141.00750357, 44527.54166392, 9141.00750357,
14059.47430713, 11450.80214932, 3986.63853774, 16705.68320797,
14059.47430713, 9141.00750357, 16705.68320797, 3986.63853774,
 9141.00750357, 9141.00750357, 9141.00750357, 25627.89524677,
 9141.00750357, 3986.63853774, 3986.63853774, 45121.15854841,
13263.79698437, 9141.00750357, 19799.50996132, 14059.47430713,
40942.4943172 , 9141.00750357, 9141.00750357, 13123.95538509,
 3986.63853774, 3986.63853774, 20436.74816106, 20436.74816106,
16705.68320797, 7796.99159733, 9141.00750357, 11450.80214932,
14059.47430713, 18915.9257486 , 20436.74816106, 9141.00750357,
18915.9257486 , 3986.63853774, 9141.00750357, 9141.00750357,
11450.80214932, 3986.63853774, 11450.80214932, 44527.54166392,
 3986.63853774, 9141.00750357, 3986.63853774, 13123.95538509,
 9141.00750357, 14059.47430713, 45121.15854841, 20436.74816106,
13263.79698437, 44527.54166392, 19799.50996132, 9141.00750357,
13263.79698437, 9141.00750357, 26988.8426   , 9141.00750357,
16705.68320797, 3986.63853774, 18915.9257486 , 5421.58225157,
25627.89524677, 9141.00750357, 7796.99159733, 40942.4943172 ,
 9141.00750357, 7796.99159733, 26988.8426   , 3986.63853774,
11450.80214932, 3986.63853774, 13263.79698437, 5421.58225157,

```
 9141.00750357,  9141.00750357, 45121.15854841, 13123.95538509,
45121.15854841, 40942.4943172 ,  9141.00750357,  9141.00750357,
 3986.63853774,  7796.99159733,  3986.63853774, 13263.79698437,
45121.15854841, 14059.47430713,  3986.63853774, 16705.68320797,
 3986.63853774, 11450.80214932,  9141.00750357,  9141.00750357,
 9141.00750357, 40942.4943172 ,  7639.2207935 ,  3986.63853774,
44527.54166392, 14059.47430713, 45121.15854841, 14059.47430713,
 9141.00750357,  5421.58225157,  3986.63853774, 13263.79698437,
 9141.00750357,  3986.63853774, 14059.47430713,  9141.00750357,
 7796.99159733,  9141.00750357,  7796.99159733, 44527.54166392,
 3986.63853774, 13123.95538509,  3986.63853774,  9141.00750357,
14059.47430713,  9141.00750357, 18915.9257486 , 14059.47430713,
16705.68320797, 16705.68320797,  3986.63853774,  9141.00750357,
19799.50996132, 11450.80214932,  9141.00750357,  3986.63853774,
40942.4943172 , 16705.68320797,  9141.00750357, 14678.67167687,
11450.80214932, 19799.50996132, 45121.15854841,  3986.63853774,
11450.80214932,  3986.63853774, 19799.50996132,  3986.63853774,
 7639.2207935 ,  3986.63853774, 14059.47430713,  9141.00750357,
14059.47430713, 45121.15854841,  9141.00750357, 40942.4943172 ,
20436.74816106, 14059.47430713, 11450.80214932, 40942.4943172 ,
13123.95538509,  7796.99159733, 13263.79698437, 25627.89524677,
40942.4943172 , 20436.74816106, 13123.95538509, 44527.54166392,
 9141.00750357, 18915.9257486 ,  9141.00750357, 40942.4943172 ,
 9141.00750357, 19530.78879845])
```

[17]:
```
from sklearn.metrics import r2_score
score =r2_score(y_test,y_pred)
score
```

[17]:
```
0.8494579300295322
```

[ ]:
```

```

DT_GRID

October 22, 2026

[70]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import pickle
```

[71]:
```
dataset = pd.read_csv("insurance.csv")
dataset
```

[71]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
…	…	…	…	…	…	…	…
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

[1338 rows x 7 columns]

[72]:
```
dataset =pd.get_dummies(dataset,drop_first= True)
dataset
```

[72]:

	age	bmi	children	charges	sex_male	smoker_yes	\
0	19	27.900	0	16884.92400	False	True	
1	18	33.770	1	1725.55230	True	False	
2	28	33.000	3	4449.46200	True	False	
3	33	22.705	0	21984.47061	True	False	
4	32	28.880	0	3866.85520	True	False	
…	…	…	…	…	…	…	
1333	50	30.970	3	10600.54830	True	False	
1334	18	31.920	0	2205.98080	False	False	
1335	18	36.850	0	1629.83350	False	False	
1336	21	25.800	0	2007.94500	False	False	
1337	61	29.070	0	29141.36030	False	True	

```
      region_northwest  region_southeast  region_southwest
0                False             False              True
1                False              True             False
2                False              True             False
3                 True             False             False
4                 True             False             False
…                 …                …                 …
1333              True             False             False
1334             False             False             False
1335             False              True             False
1336             False             False              True
1337              True             False             False

[1338 rows x 9 columns]
```

[73]:
```
x = dataset[["age","bmi","children","smoker_yes","sex_male"]]
y = dataset[["charges"]]
```

[74]:
```
y
```

[74]:
```
          charges
0     16884.92400
1      1725.55230
2      4449.46200
3     21984.47061
4      3866.85520
…             …
1333  10600.54830
1334   2205.98080
1335   1629.83350
1336   2007.94500
1337  29141.36030

[1338 rows x 1 columns]
```

[75]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 1/
 ↪3,random_state =0 )
```

[76]:
```
from sklearn.preprocessing import StandardScaler
sc= StandardScaler()
x_train =sc.fit_transform(x_train)
x_test =sc.transform(x_test)
```

[80]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor
```

```
param_grid = {
    'criterion': ['poisson', 'squared_error','absolute_error'],
    'max_features': ['auto','sqrt','log2'],
    'splitter':['best','random']
}
grid =GridSearchCV(DecisionTreeRegressor(), param_grid,refit =True,verbose␣
 ↪=3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits

C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_validation.py:493: FitFailedWarning:
30 fits failed out of a total of 90.
The score on these train-test partitions for these parameters will be set to
nan.
If these failures are not expected, you can try to debug them by setting
error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
15 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_validation.py", line 855, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py", line 1393, in wrapper
    estimator._validate_params()
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py", line 554, in _validate_params
    validate_parameter_constraints(
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\utils\_param_validation.py", line 98, in
validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features'
parameter of DecisionTreeRegressor must be an int in the range [1, inf), a float
in the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto'
instead.

--------------------------------------------------------------------------------
15 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_validation.py", line 855, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
```

```
packages\sklearn\base.py", line 1393, in wrapper
    estimator._validate_params()
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py", line 554, in _validate_params
    validate_parameter_constraints(
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\utils\_param_validation.py", line 98, in
validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features'
parameter of DecisionTreeRegressor must be an int in the range [1, inf), a float
in the range (0.0, 1.0], a str among {'sqrt', 'log2'} or None. Got 'auto'
instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_search.py:1237: UserWarning: One or more of
the test scores are non-finite: [       nan        nan 0.62289143 0.53924795
0.59078915 0.60397066
        nan        nan 0.58529629 0.50170534 0.56794655 0.52908269
        nan        nan 0.57198079 0.61345383 0.61903375 0.54928681]
  warnings.warn(
```

[80]:
```
GridSearchCV(estimator=DecisionTreeRegressor(), n_jobs=-1,
             param_grid={'criterion': ['poisson', 'squared_error',
                                       'absolute_error'],
                         'max_features': ['auto', 'sqrt', 'log2'],
                         'splitter': ['best', 'random']},
             verbose=3)
```

[81]:
```
re=grid.cv_results_
print("The R_score valu for best parameter {}:".format(grid.best_params_))
```

```
The R_score valu for best parameter {'criterion': 'poisson', 'max_features':
'sqrt', 'splitter': 'best'}:
```

[82]:
```
table=pd.DataFrame.from_dict(re)
table
```

[82]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	0.001289	0.001382	0.000000	0.000000	
1	0.002436	0.001853	0.000000	0.000000	
2	0.011710	0.001540	0.007418	0.002797	
3	0.013010	0.005841	0.006998	0.002033	
4	0.011956	0.003901	0.005291	0.001717	
5	0.012795	0.004448	0.005376	0.001588	
6	0.003245	0.002255	0.000000	0.000000	
7	0.003138	0.000937	0.000000	0.000000	

```
8        0.011870      0.001970         0.005626        0.001375
9        0.009247      0.001962         0.005054        0.000970
10       0.009504      0.001547         0.006579        0.003099
11       0.009228      0.001620         0.004608        0.000784
12       0.003373      0.000848         0.000000        0.000000
13       0.002698      0.001143         0.000000        0.000000
14       0.020823      0.004677         0.004653        0.000845
15       0.015157      0.004076         0.004034        0.000789
16       0.017112      0.002703         0.005121        0.001272
17       0.017245      0.002159         0.002910        0.001143

   param_criterion param_max_features param_splitter  \
0          poisson               auto           best
1          poisson               auto         random
2          poisson               sqrt           best
3          poisson               sqrt         random
4          poisson               log2           best
5          poisson               log2         random
6    squared_error               auto           best
7    squared_error               auto         random
8    squared_error               sqrt           best
9    squared_error               sqrt         random
10   squared_error               log2           best
11   squared_error               log2         random
12  absolute_error               auto           best
13  absolute_error               auto         random
14  absolute_error               sqrt           best
15  absolute_error               sqrt         random
16  absolute_error               log2           best
17  absolute_error               log2         random

                                               params  split0_test_score  \
0   {'criterion': 'poisson', 'max_features': 'auto…                NaN
1   {'criterion': 'poisson', 'max_features': 'auto…                NaN
2   {'criterion': 'poisson', 'max_features': 'sqrt…           0.652141
3   {'criterion': 'poisson', 'max_features': 'sqrt…           0.481385
4   {'criterion': 'poisson', 'max_features': 'log2…           0.485593
5   {'criterion': 'poisson', 'max_features': 'log2…           0.637733
6   {'criterion': 'squared_error', 'max_features':…                NaN
7   {'criterion': 'squared_error', 'max_features':…                NaN
8   {'criterion': 'squared_error', 'max_features':…           0.607436
9   {'criterion': 'squared_error', 'max_features':…           0.546447
10  {'criterion': 'squared_error', 'max_features':…           0.620229
11  {'criterion': 'squared_error', 'max_features':…           0.492710
12  {'criterion': 'absolute_error', 'max_features'…                NaN
13  {'criterion': 'absolute_error', 'max_features'…                NaN
14  {'criterion': 'absolute_error', 'max_features'…           0.551692
```

```
15  {'criterion': 'absolute_error', 'max_features'…          0.701586
16  {'criterion': 'absolute_error', 'max_features'…          0.629207
17  {'criterion': 'absolute_error', 'max_features'…          0.541289

    split1_test_score  split2_test_score  split3_test_score  \
0                 NaN                NaN                NaN
1                 NaN                NaN                NaN
2            0.585857           0.594707           0.653516
3            0.595220           0.578541           0.452106
4            0.617344           0.746254           0.513702
5            0.537149           0.641942           0.526042
6                 NaN                NaN                NaN
7                 NaN                NaN                NaN
8            0.628513           0.701122           0.623209
9            0.387921           0.624687           0.544822
10           0.474107           0.678547           0.483566
11           0.643091           0.697528           0.249680
12                NaN                NaN                NaN
13                NaN                NaN                NaN
14           0.551385           0.509480           0.657160
15           0.602708           0.659916           0.407257
16           0.541847           0.630430           0.637962
17           0.553725           0.630168           0.435621

    split4_test_score  mean_test_score  std_test_score  rank_test_score
0                 NaN              NaN             NaN               13
1                 NaN              NaN             NaN               13
2            0.628237         0.622891        0.028241                1
3            0.588989         0.539248        0.060154               10
4            0.591054         0.590789        0.091524                5
5            0.676988         0.603971        0.060748                4
6                 NaN              NaN             NaN               13
7                 NaN              NaN             NaN               13
8            0.366202         0.585296        0.114204                6
9            0.404650         0.501705        0.090942               12
10           0.583284         0.567947        0.078902                8
11           0.562405         0.529083        0.156123               11
12                NaN              NaN             NaN               13
13                NaN              NaN             NaN               13
14           0.590187         0.571981        0.049657                7
15           0.695802         0.613454        0.108940                3
16           0.655722         0.619034        0.039738                2
17           0.585632         0.549287        0.064601                9
```

[83]:
```
age_input = float(input("Age: "))
bmi_input = float(input("BMI: "))
child_input = float(input("children: "))
```

```
sex_male_input = float(input("sex male 0 or 1: "))
smoker_input = float(input("Smoker yes 0 or 1: "))
```

```
Age:  21
BMI:  30
children:  0
sex male 0 or 1:  0
Smoker yes 0 or 1:  1
```

[84]:
```
Future_Prediction=grid.
  ↪predict([[age_input,bmi_input,child_input,sex_male_input,smoker_input]])
print("Futrue Prediction={}".format(Future_Prediction))
```

```
Futrue Prediction=[11576.13]
```

[ ]:

Random_grid

October 22, 2026

[70]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import pickle
```

[71]:
```
dataset = pd.read_csv("insurance.csv")
dataset
```

[71]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
…	…	…	…	…	…	…	…
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

[1338 rows x 7 columns]

[72]:
```
dataset =pd.get_dummies(dataset,drop_first= True)
dataset
```

[72]:

	age	bmi	children	charges	sex_male	smoker_yes	\
0	19	27.900	0	16884.92400	False	True	
1	18	33.770	1	1725.55230	True	False	
2	28	33.000	3	4449.46200	True	False	
3	33	22.705	0	21984.47061	True	False	
4	32	28.880	0	3866.85520	True	False	
…	…	…	…	…	…	…	
1333	50	30.970	3	10600.54830	True	False	
1334	18	31.920	0	2205.98080	False	False	
1335	18	36.850	0	1629.83350	False	False	
1336	21	25.800	0	2007.94500	False	False	
1337	61	29.070	0	29141.36030	False	True	

```
      region_northwest  region_southeast  region_southwest
0                False             False              True
1                False              True             False
2                False              True             False
3                 True             False             False
4                 True             False             False
…                 …                …                 …
1333              True             False             False
1334             False             False             False
1335             False              True             False
1336             False             False              True
1337              True             False             False

[1338 rows x 9 columns]
```

[73]:
```
x = dataset[["age","bmi","children","smoker_yes","sex_male"]]
y = dataset[["charges"]]
```

[74]:
```
y
```

[74]:
```
          charges
0     16884.92400
1      1725.55230
2      4449.46200
3     21984.47061
4      3866.85520
…             …
1333  10600.54830
1334   2205.98080
1335   1629.83350
1336   2007.94500
1337  29141.36030

[1338 rows x 1 columns]
```

[75]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 1/
↪3,random_state =0 )
```

[85]:
```
from sklearn.preprocessing import StandardScaler
sc= StandardScaler()
x_train =sc.fit_transform(x_train)
x_test =sc.transform(x_test)
```

[88]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {
    'criterion': ['poisson', 'squared_error','absolute_error'],
    'max_features': ['auto','sqrt','log2'],
   'n_estimators':[10,100]
}
grid =GridSearchCV(RandomForestRegressor(), param_grid,refit =True,verbose␣
 ↪=3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits

C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_validation.py:493: FitFailedWarning:
30 fits failed out of a total of 90.
The score on these train-test partitions for these parameters will be set to
nan.
If these failures are not expected, you can try to debug them by setting
error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
22 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_validation.py", line 855, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py", line 1393, in wrapper
    estimator._validate_params()
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py", line 554, in _validate_params
    validate_parameter_constraints(
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\utils\_param_validation.py", line 98, in
validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features'
parameter of RandomForestRegressor must be an int in the range [1, inf), a float
in the range (0.0, 1.0], a str among {'log2', 'sqrt'} or None. Got 'auto'
instead.

--------------------------------------------------------------------------------
8 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_validation.py", line 855, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
```

```
packages\sklearn\base.py", line 1393, in wrapper
    estimator._validate_params()
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py", line 554, in _validate_params
    validate_parameter_constraints(
  File "C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\utils\_param_validation.py", line 98, in
validate_parameter_constraints
    raise InvalidParameterError(
sklearn.utils._param_validation.InvalidParameterError: The 'max_features'
parameter of RandomForestRegressor must be an int in the range [1, inf), a float
in the range (0.0, 1.0], a str among {'sqrt', 'log2'} or None. Got 'auto'
instead.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\model_selection\_search.py:1237: UserWarning: One or more of
the test scores are non-finite: [       nan        nan 0.78657436 0.79984882
0.77768039 0.80363659
        nan        nan 0.78036379 0.80034404 0.78239123 0.80041997
        nan        nan 0.7767014  0.79909805 0.78717951 0.79754119]
  warnings.warn(
C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\base.py:1403: DataConversionWarning: A column-vector y was
passed when a 1d array was expected. Please change the shape of y to
(n_samples,), for example using ravel().
  return fit_method(estimator, *args, **kwargs)
```

[88]:
```
GridSearchCV(estimator=RandomForestRegressor(), n_jobs=-1,
             param_grid={'criterion': ['poisson', 'squared_error',
                                       'absolute_error'],
                         'max_features': ['auto', 'sqrt', 'log2'],
                         'n_estimators': [10, 100]},
             verbose=3)
```

[89]:
```
re=grid.cv_results_
print("The R_score valu for best parameter {}:".format(grid.best_params_))
```

```
The R_score valu for best parameter {'criterion': 'poisson', 'max_features':
'log2', 'n_estimators': 100}:
```

[90]:
```
table=pd.DataFrame.from_dict(re)
table
```

[90]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	\
0	0.051171	0.003743	0.000000	0.000000	
1	0.030557	0.025135	0.000000	0.000000	
2	0.219447	0.003608	0.005025	0.001013	

```
3        0.398147      0.082743         0.021129        0.002492
4        0.034767      0.002169         0.004686        0.000424
5        0.298928      0.004566         0.021452        0.001094
6        0.001598      0.000808         0.000000        0.000000
7        0.001768      0.001447         0.000000        0.000000
8        0.032738      0.003145         0.004655        0.000844
9        0.273115      0.010889         0.021788        0.001249
10       0.031348      0.000458         0.006090        0.000565
11       0.274879      0.005221         0.022085        0.002073
12       0.000820      0.000765         0.000000        0.000000
13       0.001386      0.000827         0.000000        0.000000
14       0.060076      0.003363         0.005208        0.001586
15       0.546333      0.017336         0.019417        0.001342
16       0.068337      0.008505         0.005037        0.000947
17       0.494131      0.031993         0.015304        0.005038

   param_criterion param_max_features  param_n_estimators  \
0          poisson               auto                  10
1          poisson               auto                 100
2          poisson               sqrt                  10
3          poisson               sqrt                 100
4          poisson               log2                  10
5          poisson               log2                 100
6   squared_error               auto                  10
7   squared_error               auto                 100
8   squared_error               sqrt                  10
9   squared_error               sqrt                 100
10  squared_error               log2                  10
11  squared_error               log2                 100
12  absolute_error              auto                  10
13  absolute_error              auto                 100
14  absolute_error              sqrt                  10
15  absolute_error              sqrt                 100
16  absolute_error              log2                  10
17  absolute_error              log2                 100

                                               params  split0_test_score  \
0   {'criterion': 'poisson', 'max_features': 'auto…                NaN
1   {'criterion': 'poisson', 'max_features': 'auto…                NaN
2   {'criterion': 'poisson', 'max_features': 'sqrt…           0.791711
3   {'criterion': 'poisson', 'max_features': 'sqrt…           0.805975
4   {'criterion': 'poisson', 'max_features': 'log2…           0.766861
5   {'criterion': 'poisson', 'max_features': 'log2…           0.812463
6   {'criterion': 'squared_error', 'max_features':…                NaN
7   {'criterion': 'squared_error', 'max_features':…                NaN
8   {'criterion': 'squared_error', 'max_features':…           0.782601
9   {'criterion': 'squared_error', 'max_features':…           0.812911
```

10	{'criterion': 'squared_error', 'max_features':…	0.769132
11	{'criterion': 'squared_error', 'max_features':…	0.811276
12	{'criterion': 'absolute_error', 'max_features'…	NaN
13	{'criterion': 'absolute_error', 'max_features'…	NaN
14	{'criterion': 'absolute_error', 'max_features'…	0.778211
15	{'criterion': 'absolute_error', 'max_features'…	0.801599
16	{'criterion': 'absolute_error', 'max_features'…	0.774065
17	{'criterion': 'absolute_error', 'max_features'…	0.798106

	split1_test_score	split2_test_score	split3_test_score	\
0	NaN	NaN	NaN	
1	NaN	NaN	NaN	
2	0.757445	0.826997	0.812719	
3	0.766189	0.832540	0.834206	
4	0.756618	0.819648	0.794513	
5	0.765543	0.840162	0.837411	
6	NaN	NaN	NaN	
7	NaN	NaN	NaN	
8	0.765309	0.805117	0.811208	
9	0.763099	0.837139	0.820730	
10	0.759718	0.820639	0.816019	
11	0.767559	0.833099	0.830256	
12	NaN	NaN	NaN	
13	NaN	NaN	NaN	
14	0.751771	0.821646	0.795862	
15	0.771498	0.829975	0.827285	
16	0.771809	0.816850	0.816003	
17	0.774853	0.831071	0.819601	

	split4_test_score	mean_test_score	std_test_score	rank_test_score
0	NaN	NaN	NaN	13
1	NaN	NaN	NaN	13
2	0.744001	0.786574	0.031638	8
3	0.760334	0.799849	0.031563	4
4	0.750762	0.777680	0.025807	11
5	0.762604	0.803637	0.033727	1
6	NaN	NaN	NaN	13
7	NaN	NaN	NaN	13
8	0.737583	0.780364	0.026929	10
9	0.767841	0.800344	0.029567	3
10	0.746449	0.782391	0.030250	9
11	0.759910	0.800420	0.030974	2
12	NaN	NaN	NaN	13
13	NaN	NaN	NaN	13
14	0.736017	0.776701	0.030549	12
15	0.765133	0.799098	0.027091	5
16	0.757171	0.787180	0.024576	7

```
17          0.764075        0.797541        0.025479                6
```

[91]:
```
age_input = float(input("Age: "))
bmi_input = float(input("BMI: "))
child_input = float(input("children: "))
sex_male_input = float(input("sex male 0 or 1: "))
smoker_input = float(input("Smoker yes 0 or 1: "))
```

```
Age:  21
BMI:  20
children:  0
sex male 0 or 1:  1
Smoker yes 0 or 1:  0
```

[92]:
```
Future_Prediction=grid.
  ↪predict([[age_input,bmi_input,child_input,sex_male_input,smoker_input]])
print("Futrue Prediction={}".format(Future_Prediction))
```

```
Futrue Prediction=[48418.1130422]
```

[ ]:

Svr_grid

October 22, 2026

[29]:
```
import pandas as pd
import numpy as np
import pickle
```

[30]:
```
dataset = pd.read_csv("insurance.csv")
dataset
```

[30]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
…	…	…	…	…	…	…	…
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

[1338 rows x 7 columns]

[31]:
```
dataset =pd.get_dummies(dataset,drop_first= True)
dataset
```

[31]:

	age	bmi	children	charges	sex_male	smoker_yes	\
0	19	27.900	0	16884.92400	False	True	
1	18	33.770	1	1725.55230	True	False	
2	28	33.000	3	4449.46200	True	False	
3	33	22.705	0	21984.47061	True	False	
4	32	28.880	0	3866.85520	True	False	
…	…	…	…	…	…	…	
1333	50	30.970	3	10600.54830	True	False	
1334	18	31.920	0	2205.98080	False	False	
1335	18	36.850	0	1629.83350	False	False	
1336	21	25.800	0	2007.94500	False	False	
1337	61	29.070	0	29141.36030	False	True	

```
      region_northwest  region_southeast  region_southwest
0                False             False              True
1                False              True             False
2                False              True             False
3                 True             False             False
4                 True             False             False
…                 …                …                 …
1333              True             False             False
1334             False             False             False
1335             False              True             False
1336             False             False              True
1337              True             False             False

[1338 rows x 9 columns]
```

[32]:
```
x = dataset[["age","bmi","children","smoker_yes","sex_male"]]
y = dataset[["charges"]]
```

[33]:
```
y
```

[33]:
```
          charges
0     16884.92400
1      1725.55230
2      4449.46200
3     21984.47061
4      3866.85520
…             …
1333  10600.54830
1334   2205.98080
1335   1629.83350
1336   2007.94500
1337  29141.36030

[1338 rows x 1 columns]
```

[34]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y, test_size = 1/
 ↪3,random_state =0 )
```

[35]:
```
from sklearn.preprocessing import StandardScaler
sc= StandardScaler()
x_train =sc.fit_transform(x_train)
x_test =sc.transform(x_test)
```

[39]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVR
```

```
param_grid={'kernel' :['rbf','poly','sigmoid','linear'],'C':
  ↪[10,100,1000,2000,3000],'gamma':['auto','scale']}
grid =GridSearchCV(SVR(), param_grid,refit =True,verbose =3,n_jobs=-1)
grid.fit(x_train,y_train)
```

```
Fitting 5 folds for each of 40 candidates, totalling 200 fits

C:\Users\karth\AppData\Local\Programs\Python\Python312\Lib\site-
packages\sklearn\utils\validation.py:1368: DataConversionWarning: A column-
vector y was passed when a 1d array was expected. Please change the shape of y
to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

[39]:
```
GridSearchCV(estimator=SVR(), n_jobs=-1,
             param_grid={'C': [10, 100, 1000, 2000, 3000],
                         'gamma': ['auto', 'scale'],
                         'kernel': ['rbf', 'poly', 'sigmoid', 'linear']},
             verbose=3)
```

[40]:
```
re=grid.cv_results_
print("The R_score valu for best parameter {}:".format(grid.best_params_))
```

```
The R_score valu for best parameter {'C': 3000, 'gamma': 'scale', 'kernel':
'poly'}:
```

[41]:
```
table=pd.DataFrame.from_dict(re)
table
```

[41]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	\
0	0.167120	0.003398	0.017691	0.000717	10	
1	0.105361	0.063394	0.007120	0.000964	10	
2	0.039827	0.002453	0.008480	0.001038	10	
3	0.026568	0.000834	0.005554	0.001239	10	
4	0.037627	0.001800	0.016976	0.001166	10	
5	0.029691	0.001422	0.005969	0.000118	10	
6	0.039336	0.002011	0.007871	0.000353	10	
7	0.028668	0.002103	0.006180	0.000976	10	
8	0.036682	0.002119	0.015731	0.001005	100	
9	0.031403	0.001307	0.005866	0.000753	100	
10	0.042401	0.003972	0.008275	0.000311	100	
11	0.033341	0.002464	0.005871	0.001353	100	
12	0.035452	0.001528	0.015454	0.000505	100	
13	0.030974	0.001070	0.006454	0.001068	100	
14	0.041249	0.002906	0.008474	0.001278	100	
15	0.035595	0.001649	0.004789	0.000949	100	
16	0.048703	0.002477	0.015707	0.000907	1000	
17	0.045460	0.004586	0.005961	0.001308	1000	
18	0.045981	0.003639	0.008149	0.000521	1000	
19	0.043908	0.003420	0.004437	0.000510	1000	

20	0.048207	0.001730	0.016164	0.000931	1000
21	0.045500	0.002501	0.005550	0.001101	1000
22	0.044721	0.001641	0.008093	0.000026	1000
23	0.042923	0.001548	0.005578	0.001798	1000
24	0.052918	0.001839	0.015283	0.000713	2000
25	0.100863	0.042448	0.005386	0.000783	2000
26	0.043950	0.004055	0.008764	0.000893	2000
27	0.054267	0.004800	0.005646	0.000771	2000
28	0.052929	0.001022	0.016047	0.000305	2000
29	0.123946	0.109528	0.006018	0.001046	2000
30	0.046102	0.001879	0.009272	0.000840	2000
31	0.059115	0.005187	0.006552	0.000640	2000
32	0.061273	0.003530	0.016917	0.002541	3000
33	0.206961	0.191662	0.004267	0.002736	3000
34	0.050093	0.002051	0.009919	0.001263	3000
35	0.073499	0.015735	0.005543	0.000769	3000
36	0.064015	0.003202	0.017525	0.000850	3000
37	0.120376	0.040447	0.006316	0.000692	3000
38	0.053857	0.002375	0.009707	0.001021	3000
39	0.069337	0.011569	0.004130	0.001460	3000

	param_gamma	param_kernel	\
0	auto	rbf	
1	auto	poly	
2	auto	sigmoid	
3	auto	linear	
4	scale	rbf	
5	scale	poly	
6	scale	sigmoid	
7	scale	linear	
8	auto	rbf	
9	auto	poly	
10	auto	sigmoid	
11	auto	linear	
12	scale	rbf	
13	scale	poly	
14	scale	sigmoid	
15	scale	linear	
16	auto	rbf	
17	auto	poly	
18	auto	sigmoid	
19	auto	linear	
20	scale	rbf	
21	scale	poly	
22	scale	sigmoid	
23	scale	linear	
24	auto	rbf	

```
25       auto         poly
26       auto      sigmoid
27       auto       linear
28      scale          rbf
29      scale         poly
30      scale      sigmoid
31      scale       linear
32       auto          rbf
33       auto         poly
34       auto      sigmoid
35       auto       linear
36      scale          rbf
37      scale         poly
38      scale      sigmoid
39      scale       linear
```

```
                                                 params  split0_test_score  \
0         {'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}          -0.004176
1        {'C': 10, 'gamma': 'auto', 'kernel': 'poly'}           0.047420
2     {'C': 10, 'gamma': 'auto', 'kernel': 'sigmoid'}           0.044787
3      {'C': 10, 'gamma': 'auto', 'kernel': 'linear'}           0.387624
4        {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}          -0.003956
5       {'C': 10, 'gamma': 'scale', 'kernel': 'poly'}           0.043648
6    {'C': 10, 'gamma': 'scale', 'kernel': 'sigmoid'}           0.043946
7     {'C': 10, 'gamma': 'scale', 'kernel': 'linear'}           0.387624
8        {'C': 100, 'gamma': 'auto', 'kernel': 'rbf'}           0.303414
9       {'C': 100, 'gamma': 'auto', 'kernel': 'poly'}           0.542212
10   {'C': 100, 'gamma': 'auto', 'kernel': 'sigmoid'}           0.492088
11    {'C': 100, 'gamma': 'auto', 'kernel': 'linear'}           0.596232
12      {'C': 100, 'gamma': 'scale', 'kernel': 'rbf'}           0.304939
13     {'C': 100, 'gamma': 'scale', 'kernel': 'poly'}           0.532310
14  {'C': 100, 'gamma': 'scale', 'kernel': 'sigmoid'}           0.491855
15   {'C': 100, 'gamma': 'scale', 'kernel': 'linear'}           0.596232
16      {'C': 1000, 'gamma': 'auto', 'kernel': 'rbf'}           0.731430
17     {'C': 1000, 'gamma': 'auto', 'kernel': 'poly'}           0.799185
18  {'C': 1000, 'gamma': 'auto', 'kernel': 'sigmoid'}           0.232428
19   {'C': 1000, 'gamma': 'auto', 'kernel': 'linear'}           0.686126
20     {'C': 1000, 'gamma': 'scale', 'kernel': 'rbf'}           0.732079
21    {'C': 1000, 'gamma': 'scale', 'kernel': 'poly'}           0.798212
22  {'C': 1000, 'gamma': 'scale', 'kernel': 'sigmo…           0.248266
23  {'C': 1000, 'gamma': 'scale', 'kernel': 'linear'}           0.686126
24      {'C': 2000, 'gamma': 'auto', 'kernel': 'rbf'}           0.788746
25     {'C': 2000, 'gamma': 'auto', 'kernel': 'poly'}           0.804603
26  {'C': 2000, 'gamma': 'auto', 'kernel': 'sigmoid'}          -0.656854
27   {'C': 2000, 'gamma': 'auto', 'kernel': 'linear'}           0.669958
28     {'C': 2000, 'gamma': 'scale', 'kernel': 'rbf'}           0.789144
29    {'C': 2000, 'gamma': 'scale', 'kernel': 'poly'}           0.804708
```

```
30  {'C': 2000, 'gamma': 'scale', 'kernel': 'sigmo…          -0.370980
31  {'C': 2000, 'gamma': 'scale', 'kernel': 'linear'}           0.669958
32      {'C': 3000, 'gamma': 'auto', 'kernel': 'rbf'}           0.795104
33     {'C': 3000, 'gamma': 'auto', 'kernel': 'poly'}           0.804915
34  {'C': 3000, 'gamma': 'auto', 'kernel': 'sigmoid'}          -1.795299
35   {'C': 3000, 'gamma': 'auto', 'kernel': 'linear'}           0.669819
36     {'C': 3000, 'gamma': 'scale', 'kernel': 'rbf'}           0.795353
37    {'C': 3000, 'gamma': 'scale', 'kernel': 'poly'}           0.805169
38  {'C': 3000, 'gamma': 'scale', 'kernel': 'sigmo…          -1.651895
39  {'C': 3000, 'gamma': 'scale', 'kernel': 'linear'}           0.669819

    split1_test_score  split2_test_score  split3_test_score  \
0            0.022594          -0.118956          -0.082926
1            0.077536          -0.060527          -0.009476
2            0.081689          -0.072355          -0.027541
3            0.461268           0.288301           0.340540
4            0.022453          -0.119035          -0.082925
5            0.079780          -0.059229          -0.009498
6            0.082230          -0.072132          -0.027546
7            0.461268           0.288301           0.340540
8            0.319385           0.155546           0.208414
9            0.566743           0.471172           0.537557
10           0.545107           0.438714           0.468576
11           0.635776           0.566816           0.588799
12           0.318480           0.155033           0.208422
13           0.571466           0.474948           0.537506
14           0.544604           0.439299           0.468556
15           0.635776           0.566816           0.588799
16           0.716349           0.686476           0.710172
17           0.786769           0.821307           0.810704
18           0.306815           0.236688           0.291707
19           0.598398           0.586104           0.573860
20           0.715992           0.686057           0.710177
21           0.787019           0.823118           0.810712
22           0.377692           0.315321           0.291779
23           0.598398           0.586104           0.573860
24           0.764261           0.818616           0.794131
25           0.784782           0.847674           0.807280
26          -0.432695          -0.346330          -0.484052
27           0.592446           0.586953           0.572862
28           0.764022           0.818319           0.794133
29           0.784713           0.847781           0.807295
30          -0.421011          -0.350248          -0.483750
31           0.592446           0.586953           0.572862
32           0.772917           0.846181           0.811601
33           0.783304           0.852190           0.810160
34          -1.629662          -1.461919          -1.712710
```

35	0.588893	0.587076	0.572849
36	0.772801	0.846034	0.811601
37	0.783163	0.852345	0.810160
38	-2.010670	-1.468842	-1.712109
39	0.588893	0.587076	0.572849

	split4_test_score	mean_test_score	std_test_score	rank_test_score
0	-0.103473	-0.057387	0.056205	35
1	-0.050823	0.000826	0.054025	32
2	-0.051470	-0.004978	0.058648	34
3	0.297825	0.355112	0.063693	25
4	-0.103510	-0.057395	0.056230	36
5	-0.050317	0.000877	0.053658	31
6	-0.051337	-0.004968	0.058595	33
7	0.297825	0.355112	0.063693	25
8	0.161756	0.229703	0.069348	29
9	0.413719	0.506281	0.056081	22
10	0.425516	0.474000	0.042447	23
11	0.537415	0.585008	0.032600	19
12	0.161488	0.229672	0.069604	30
13	0.415264	0.506299	0.055077	21
14	0.424226	0.473708	0.042461	24
15	0.537415	0.585008	0.032600	19
16	0.613293	0.691544	0.041718	11
17	0.719528	0.787499	0.035885	8
18	0.346868	0.282901	0.043415	28
19	0.556109	0.600119	0.045217	13
20	0.613122	0.691485	0.041876	12
21	0.719979	0.787808	0.036003	7
22	0.346175	0.315846	0.044478	27
23	0.556109	0.600119	0.045217	13
24	0.709676	0.775086	0.036986	9
25	0.745448	0.797957	0.033268	4
26	-0.076409	-0.399268	0.190629	38
27	0.556109	0.595666	0.039218	15
28	0.709586	0.775041	0.036992	10
29	0.745732	0.798046	0.033220	3
30	-0.087414	-0.342681	0.135687	37
31	0.556109	0.595666	0.039218	15
32	0.725697	0.790300	0.040188	5
33	0.748175	0.799749	0.034104	2
34	-0.929359	-1.505790	0.308601	39
35	0.557014	0.595130	0.039068	17
36	0.725615	0.790281	0.040189	6
37	0.748201	0.799808	0.034165	1
38	-0.961821	-1.561068	0.346643	40
39	0.557014	0.595130	0.039068	17

[45]:
```
age_input = float(input("Age: "))
bmi_input = float(input("BMI: "))
child_input = float(input("children: "))
sex_male_input = float(input("sex male 0 or 1: "))
smoker_input = float(input("Smoker yes 0 or 1: "))
```

```
Age:  21
BMI:  30
children:  0
sex male 0 or 1:  0
Smoker yes 0 or 1:  1
```

[46]:
```
Future_Prediction=grid.
  ↪predict([[age_input,bmi_input,child_input,sex_male_input,smoker_input]])
print("Futrue Prediction={}".format(Future_Prediction))
```

```
Futrue Prediction=[1112705.10012247]
```

[ ]:
```

```